

Chapter 14. On to Object Design

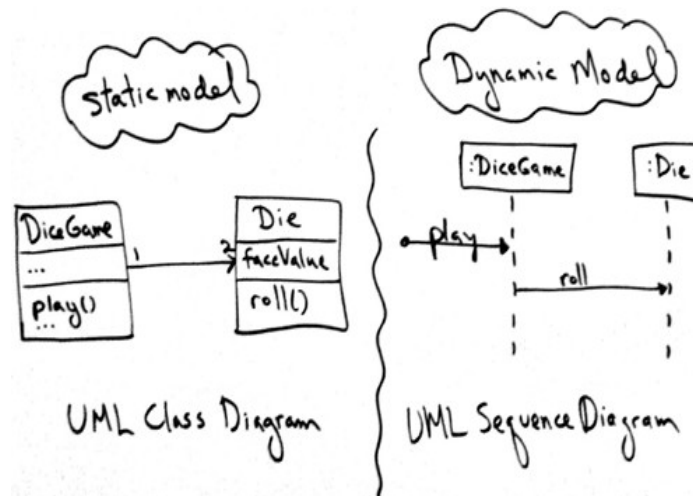
“I do not like this word ‘bomb.’ It is not a bomb. It is a device that is exploding.”

Objectives

- Understand dynamic and static object modeling.
- Try agile modeling, or a UML CASE tool for drawing.

Designing Objects: Static vs. Dynamic

- **Dynamic models:** design the logic, the behavior of the code or the method bodies.
 - e.g. UML interaction diagrams (sequence diagrams or communication diagrams)
 - Tend to be the more interesting, difficult, and important diagrams to create.
- **Static models:** design the definition of packages, class names, attributes, and method signatures (but not method bodies).
 - e.g. UML class diagrams
- Agile modeling practice
 - Spend a short period of time on interaction diagrams (dynamics), then switch to a wall of related class diagrams (statics).



Object Design Skill over UML Notation Skill

- **Drawing UML** is a reflection of making decisions about the design.
- **The object design skills** are what matter, not knowing how to draw UML.
- **Fundamental object design** requires knowledge of:
 - Principles of responsibility assignment
 - Design patterns

Chapter 15. UML Interaction Diagrams

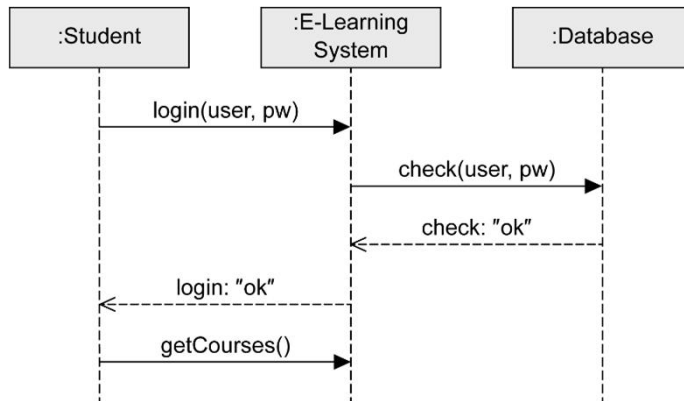
“Cats are smarter than dogs. You can’t get eight cats to pull a sled through snow.”

Objectives

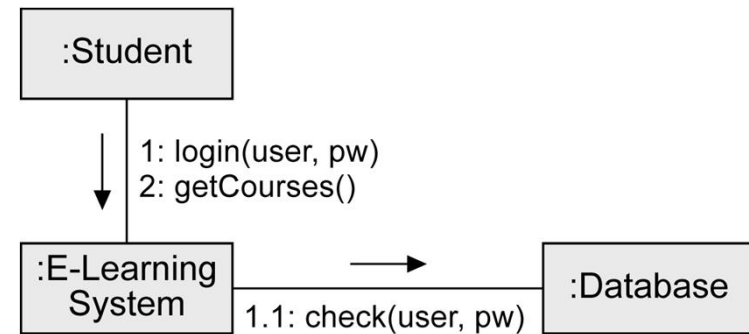
- Provide a reference for frequently used UML interaction diagram notations - sequence and communication diagrams.

Interaction Diagrams

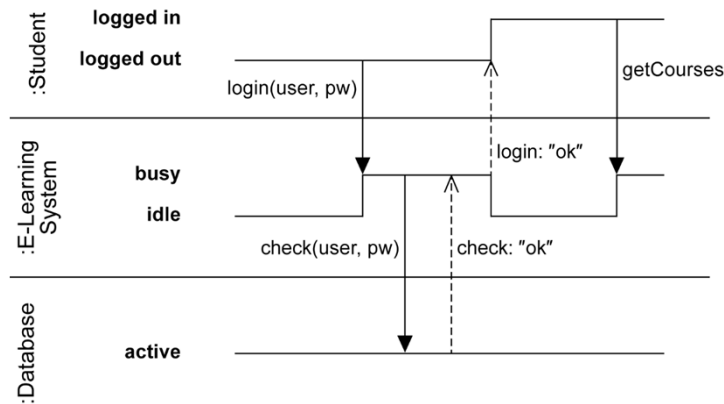
- illustrate how objects interact via messages → **dynamic object modeling**



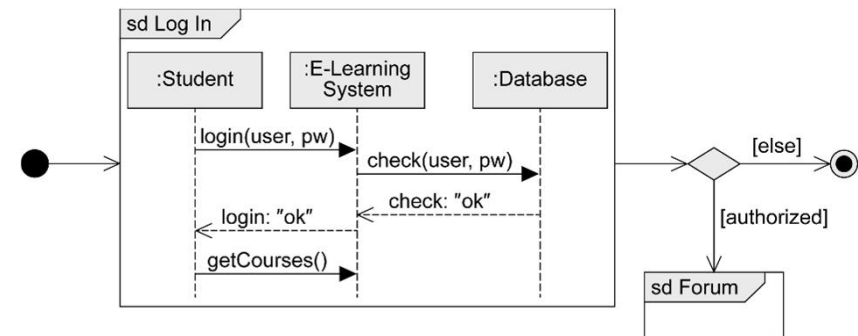
Sequence diagram



Communication diagram



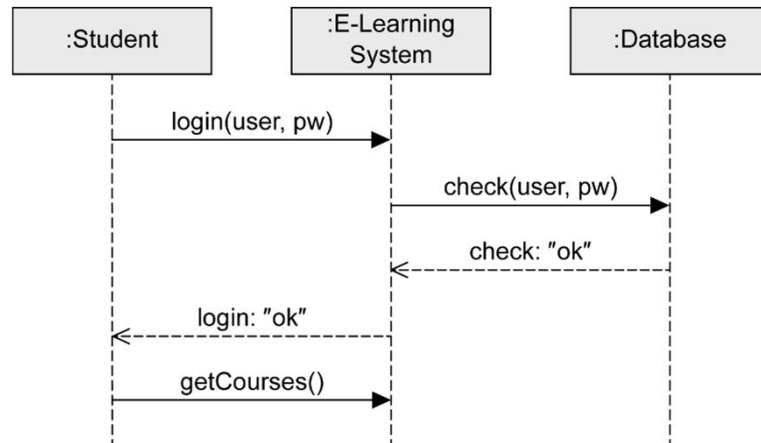
Timing diagram



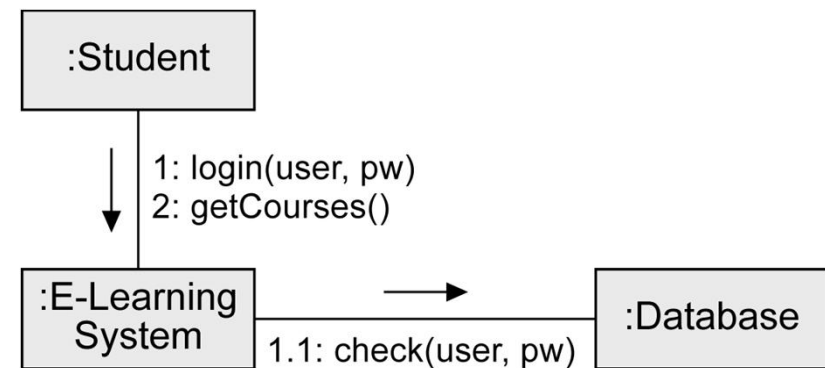
Interaction Overview diagram

Sequence Diagram = Communication Diagram

- Equivalent, but different focus

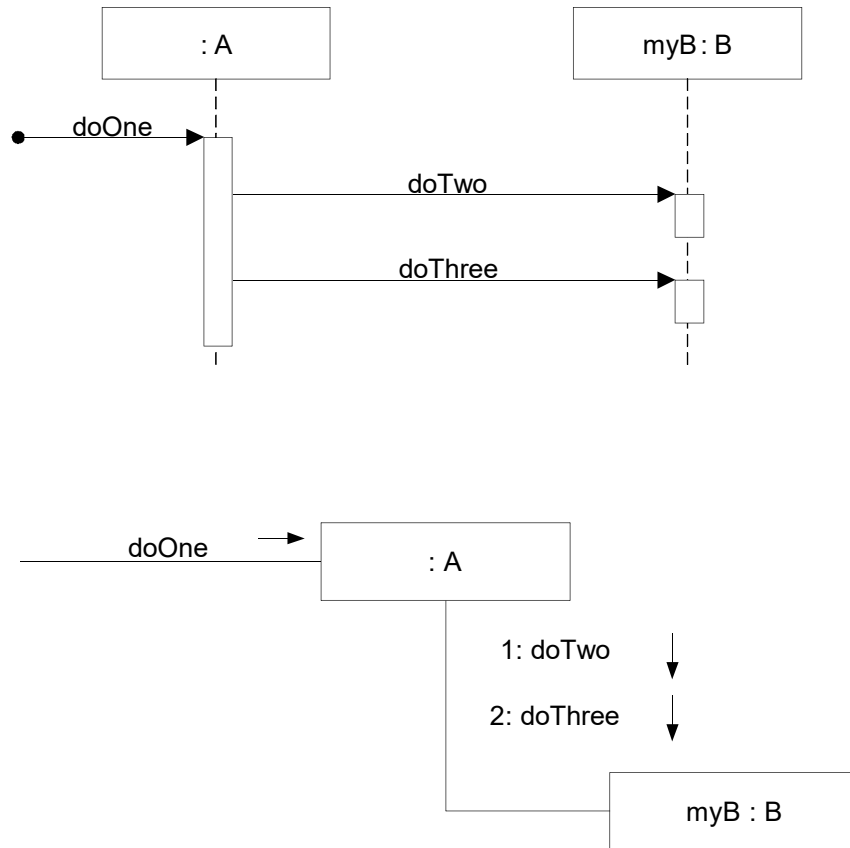


- model the collaboration of objects based on a time sequence
- focus: chronological order of interactions



- models the relationships between communication partners
- focus: Who communicates with whom
- **message order** via **decimal classification**
- space-efficient
 - better for hand-writing discussions

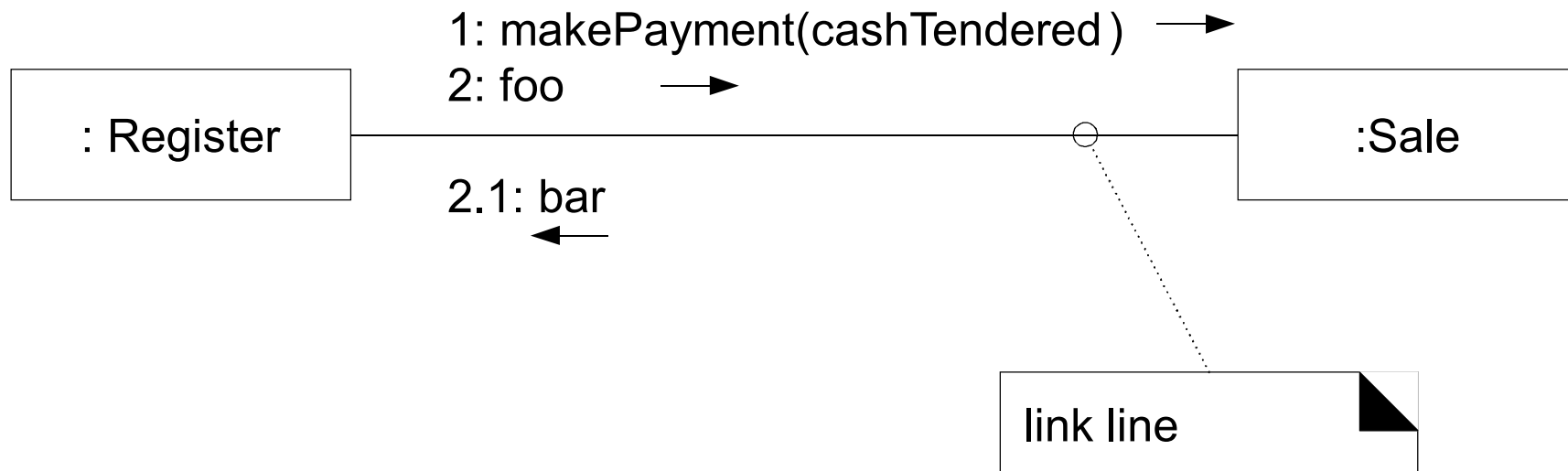
Sequence Diagram = Communication Diagram



```
public class A {  
    B myB;  
    // other code here ...  
    public void doOne() {  
        myB.doTwo();  
        myB.doThree();  
    }  
}
```

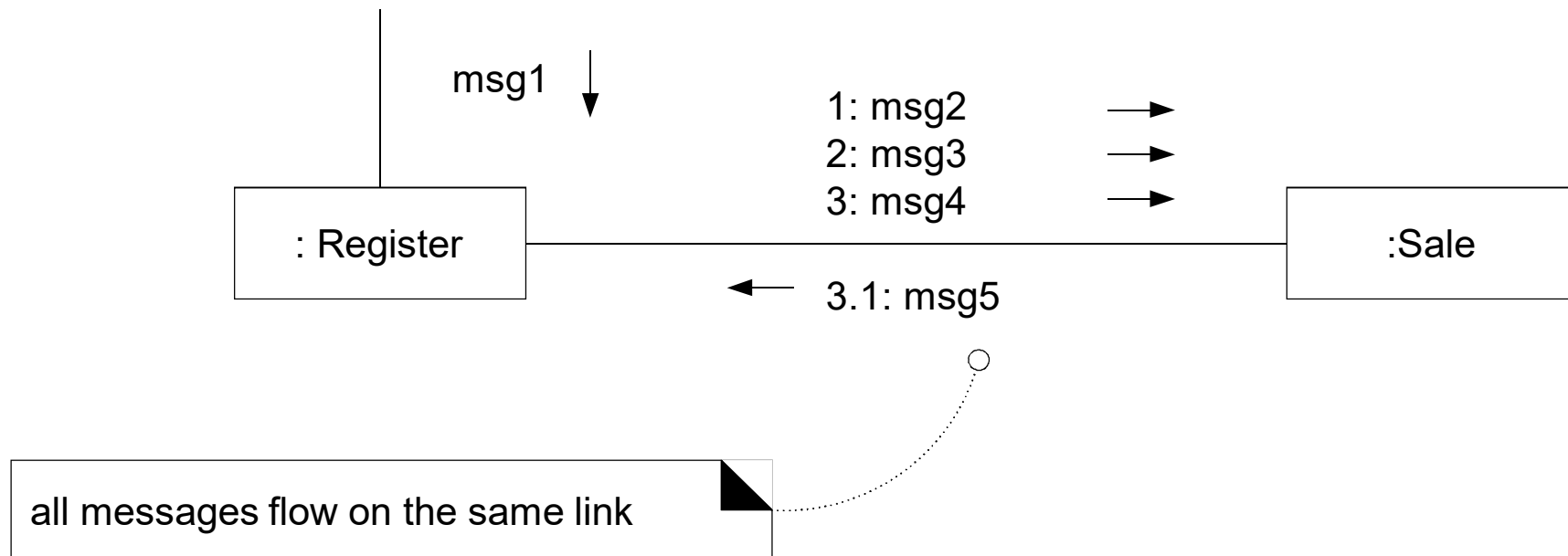
Communication Diagram: Link

- Link
 - a **connection path** between two objects
 - indicates some **form of navigation and visibility** between the objects is possible
 - More formally, a link is an **instance of an association**, dependency and aggregation (**but not inheritance!**)
 - e.g. there is a link or path of navigation **from a Register to a Sale**, along which **messages may flow**, such as the makePayment message.



Communication Diagram: Message

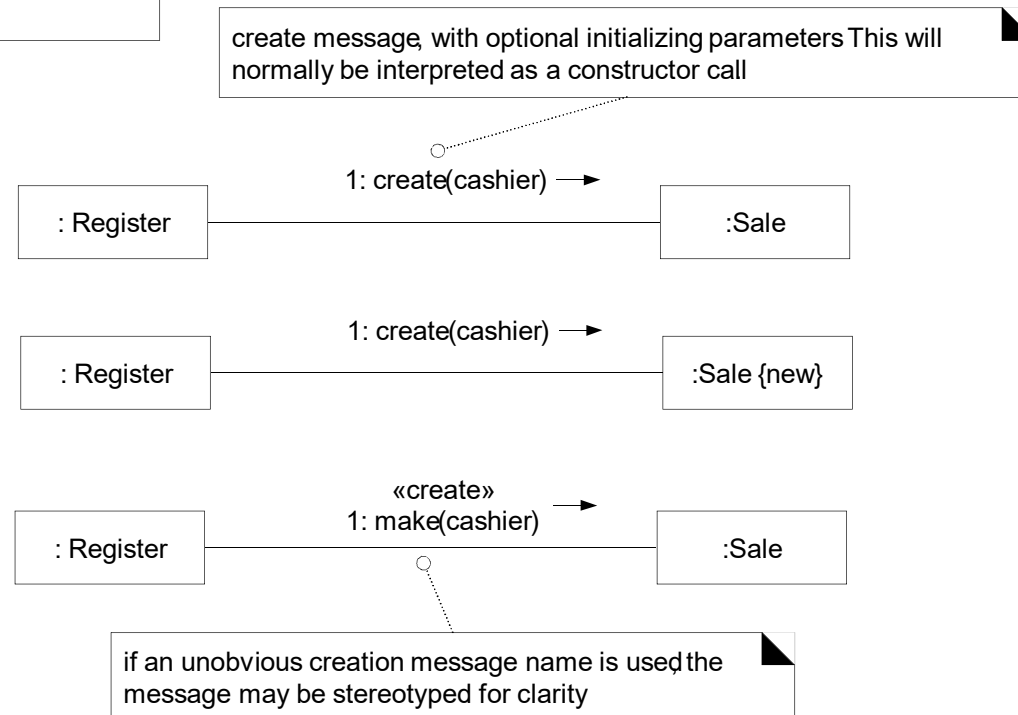
- Messages
 - Each message between objects is represented with a message expression and small arrow indicating the direction of the message
 - Many messages may flow along a link
 - A sequence number is added to show the sequential order of messages in the current thread of control.



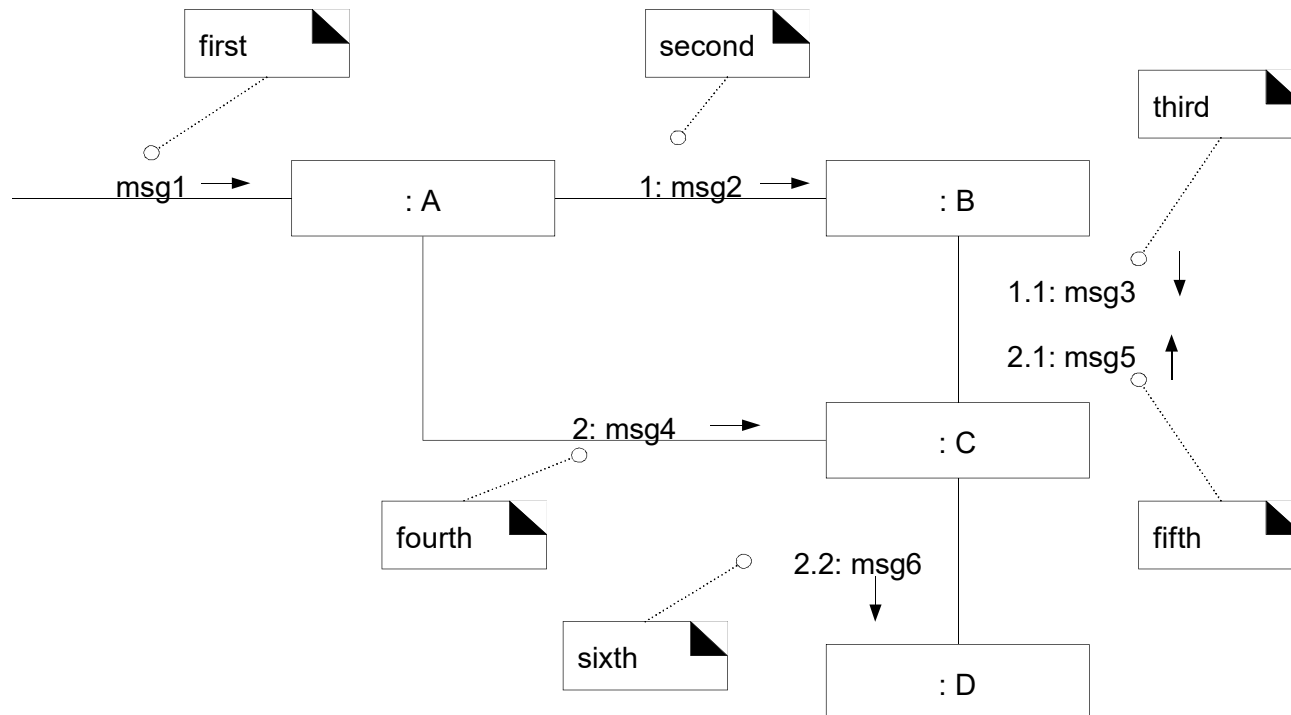
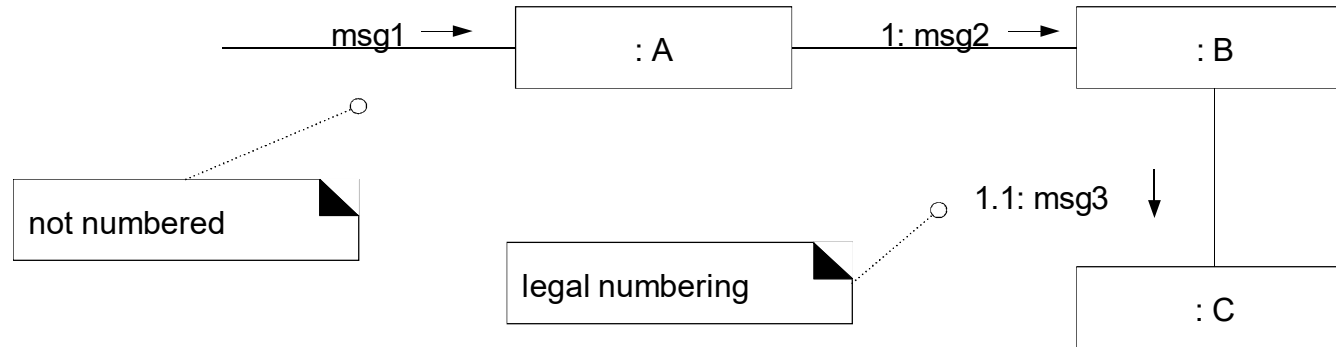
Communication Diagram: Creation

- Any message can be used to create an instance
 - Use a message named **create** for this purpose (some use **new**)
 - UML tagged value **{new}** may optionally be added to the lifeline box to highlight the creation.
 - If another (less obvious) message name is used, the message may be annotated with a UML stereotype, like **«create»**

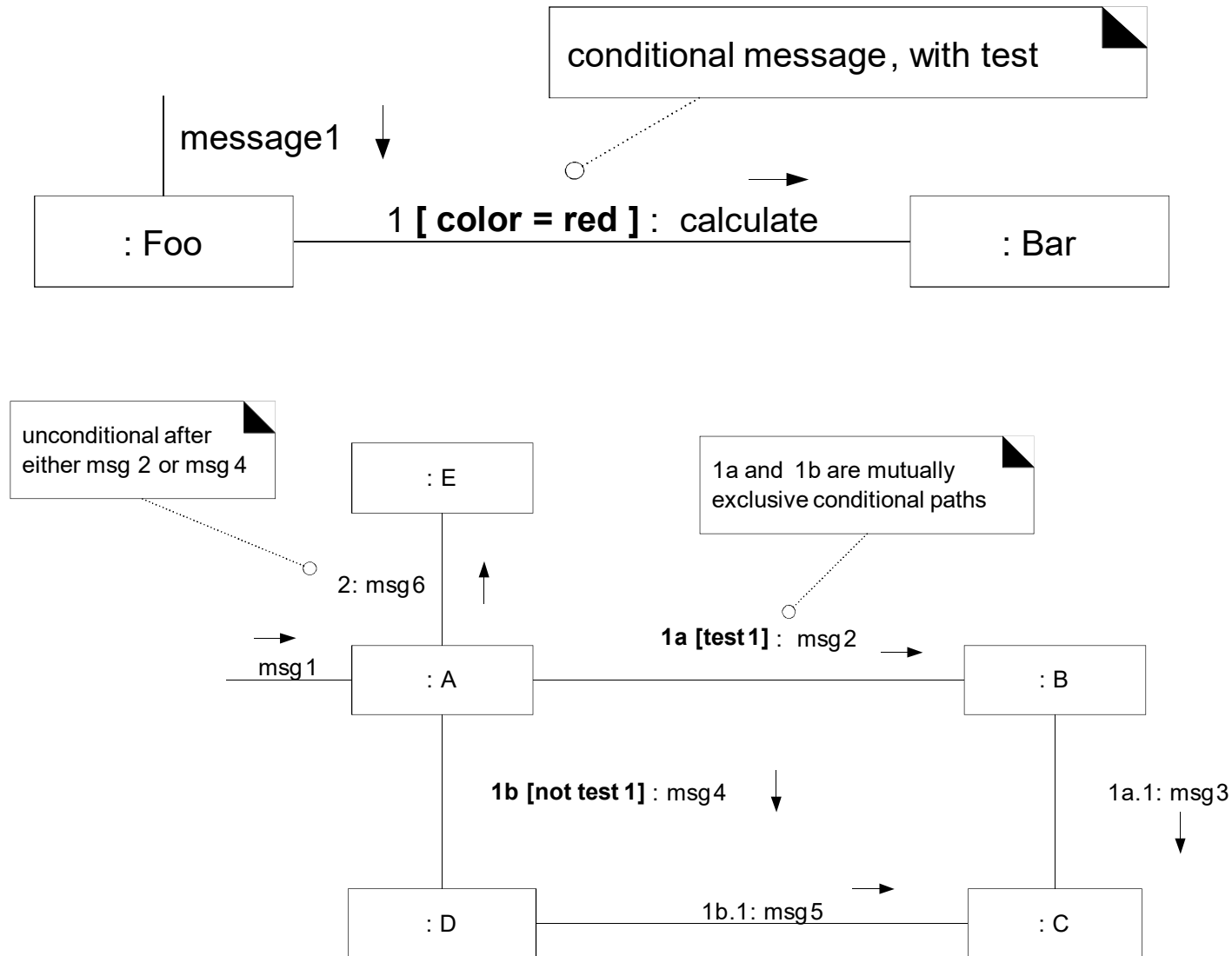
Three ways to show creation in a communication diagram



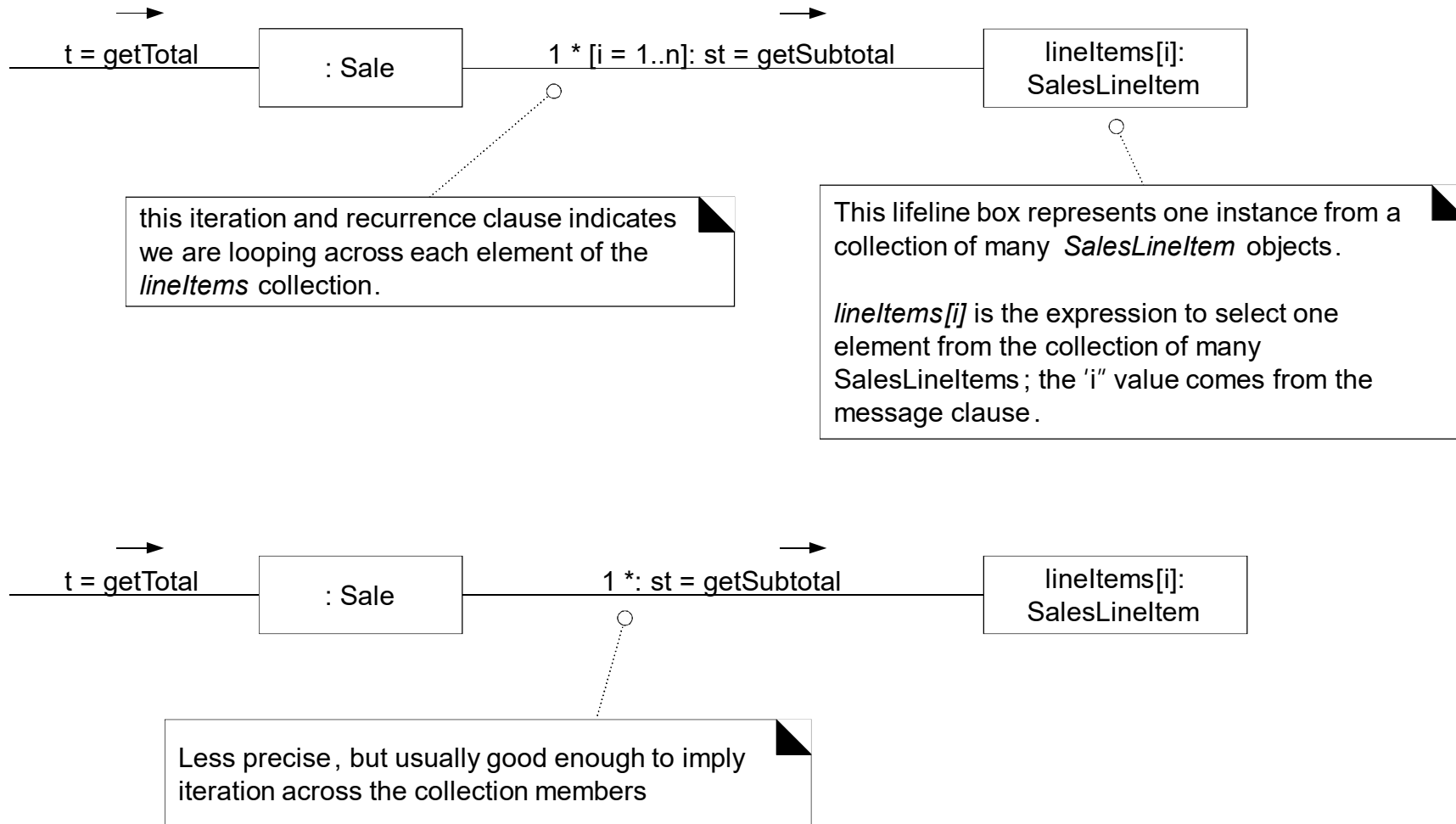
Communication Diagram: Numbering



Communication Diagram: Conditional / Mutually Exclusive messages

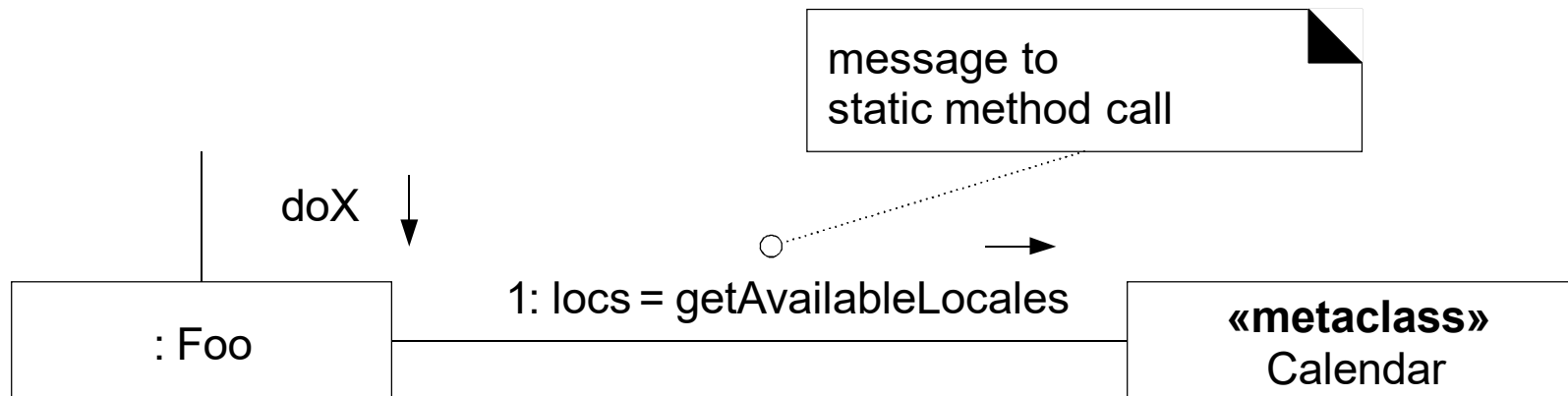


Communication Diagram: Iterative messages

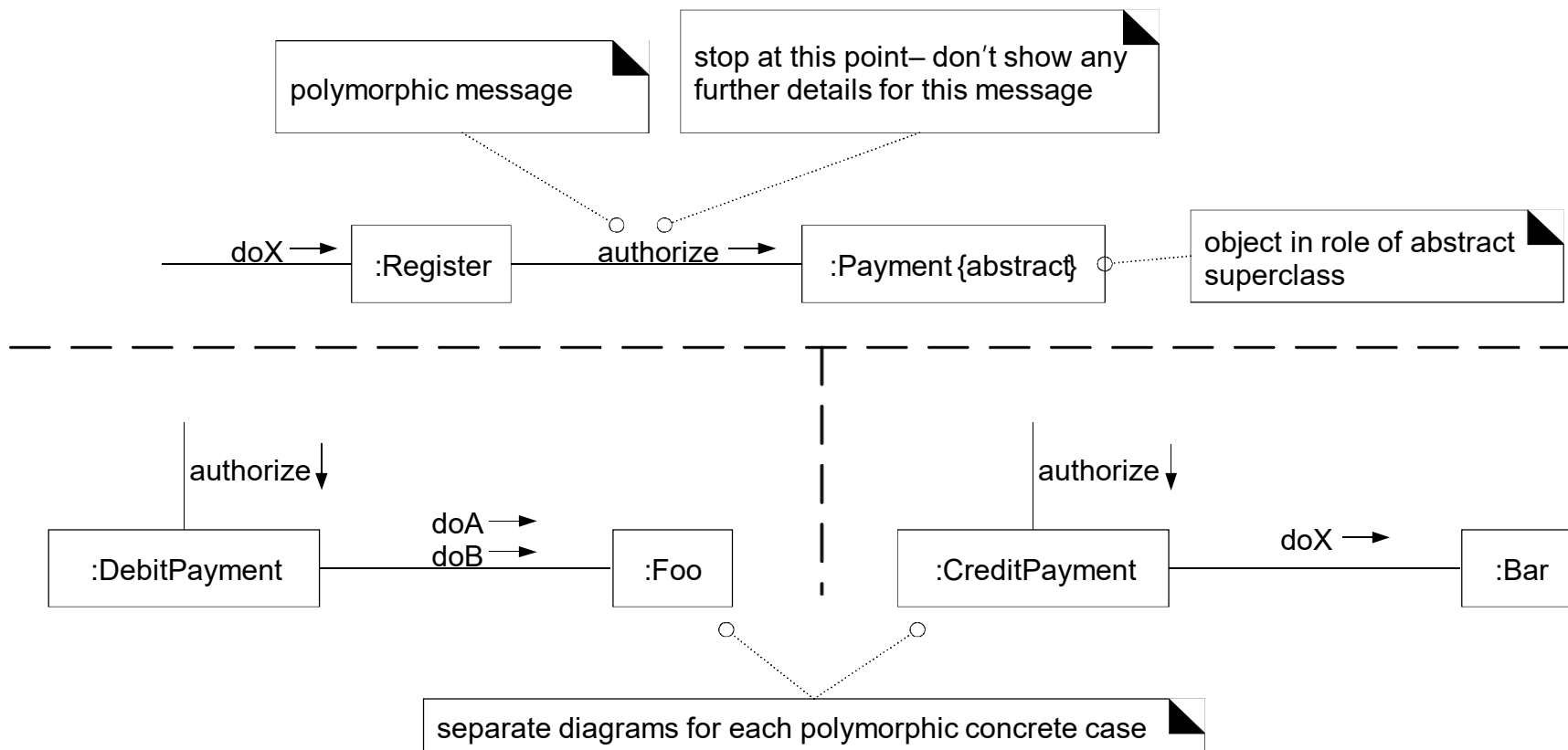


Communication Diagram: Static Method call

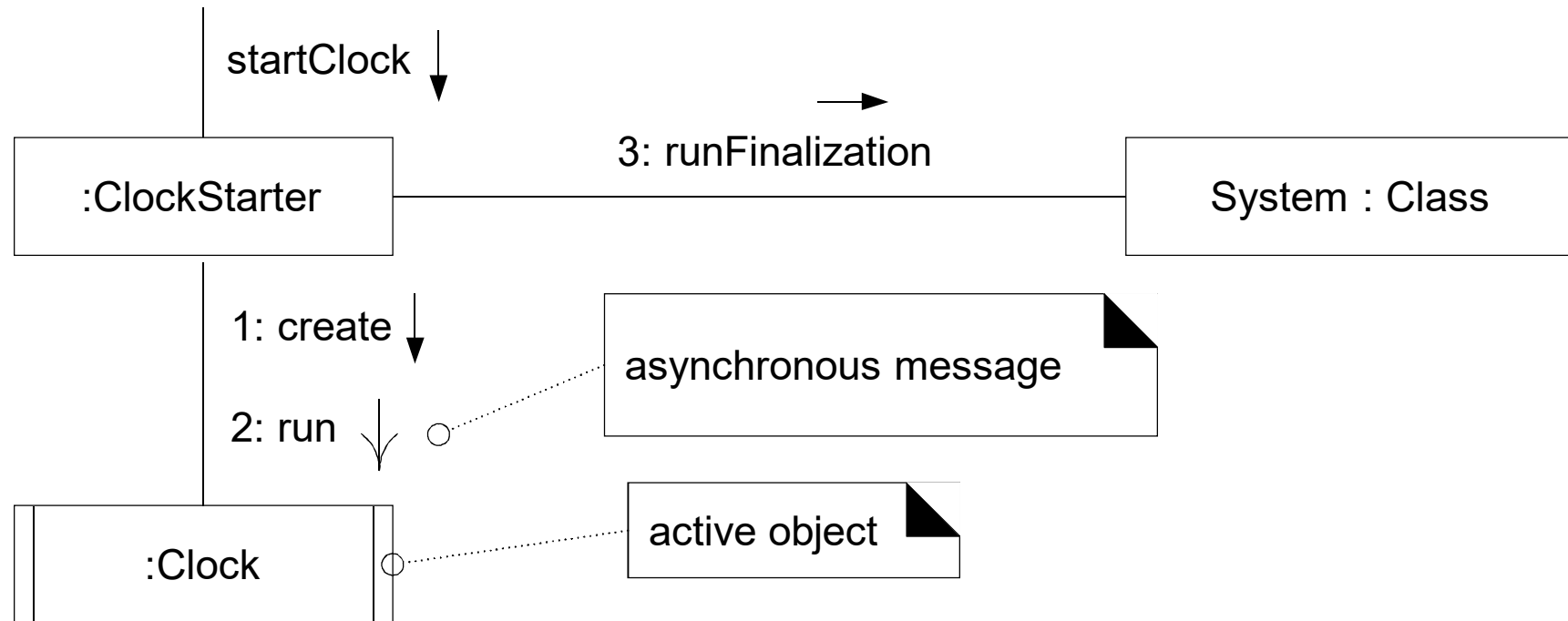
- You can show class or static method calls by
 - using a lifeline box label that indicates the receiving object is a class, or more precisely, an instance of a metaclass



Communication Diagram: Polymorphic Method Call

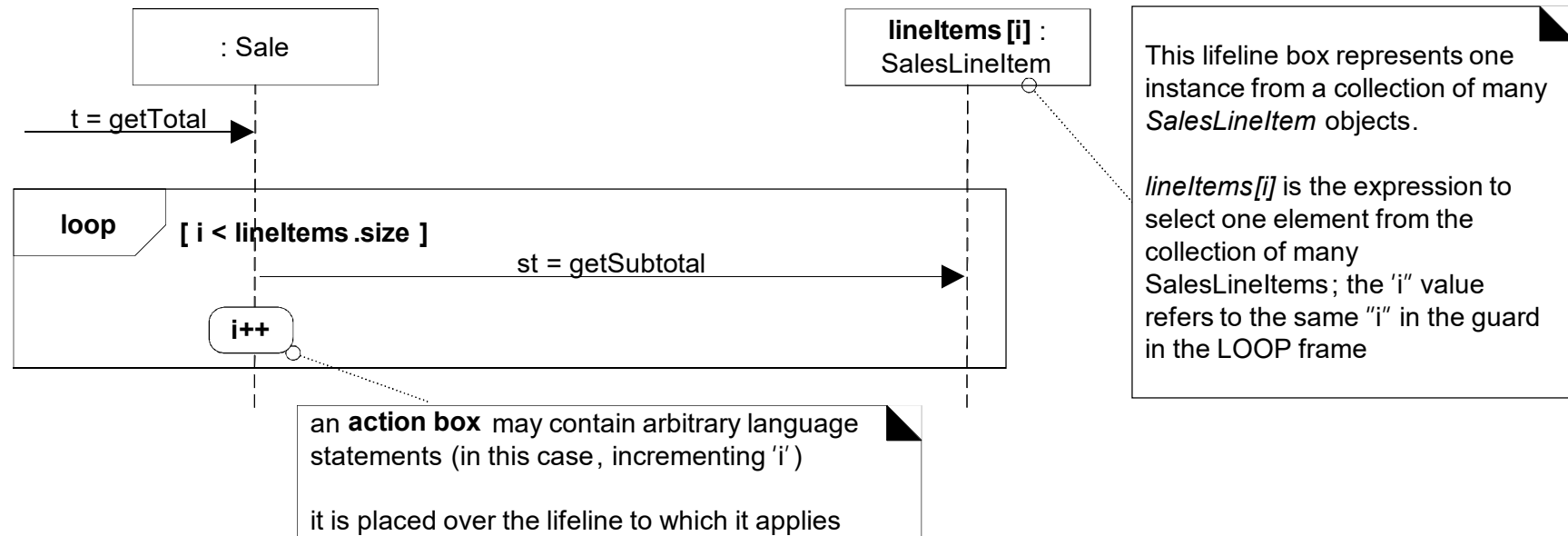


Communication Diagram: Asynchronous and Synchronous Calls



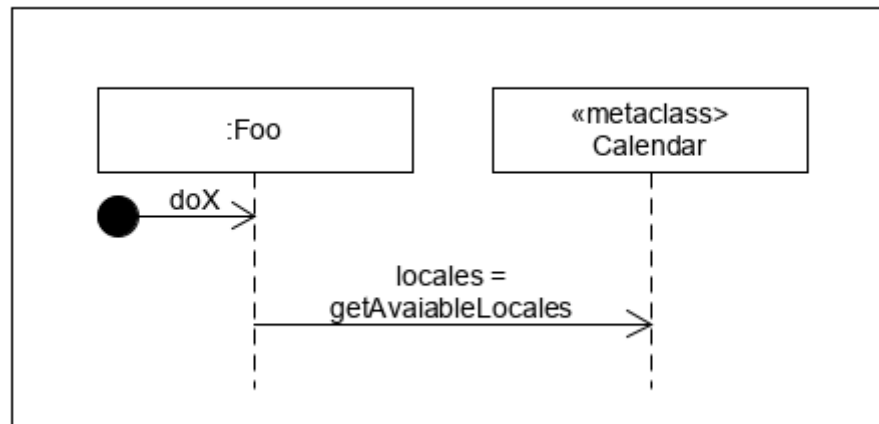
Sequence Diagram: Iteration over a Collection

- Sending the same message to each object to iterate over all members of a collection (such as a list or map).
 - The selector expression, e.g. `lineltems[i]`, selects one object from a group.
 - Lifeline participants should represent one object, not a collection.



Sequence Diagram: Messages to Classes to Invoke Static Methods

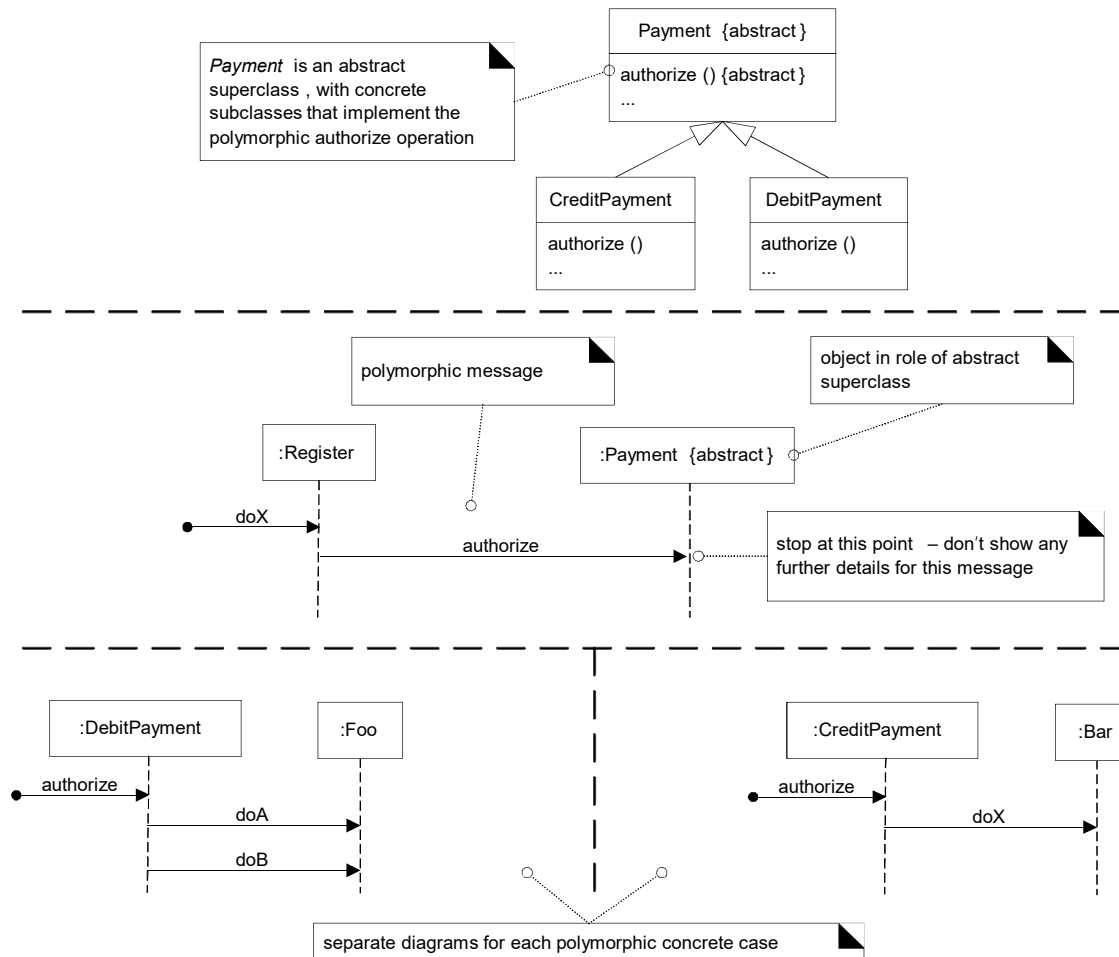
- You can show class or static method calls by
 - using a lifeline box label that indicates the receiving object is a class, or more precisely, an instance of a metaclass



Sequence Diagram

Polymorphic Method Call

- Shows the **polymorphic message** to the **abstract superclass** or **interface object**,
- and then **separate** sequence diagrams detailing **each polymorphic case** starting with a **found polymorphic message**.



Sequence Diagram

Asynchronous and Synchronous Calls

a **stick arrow** in UML implies an **asynchronous call**

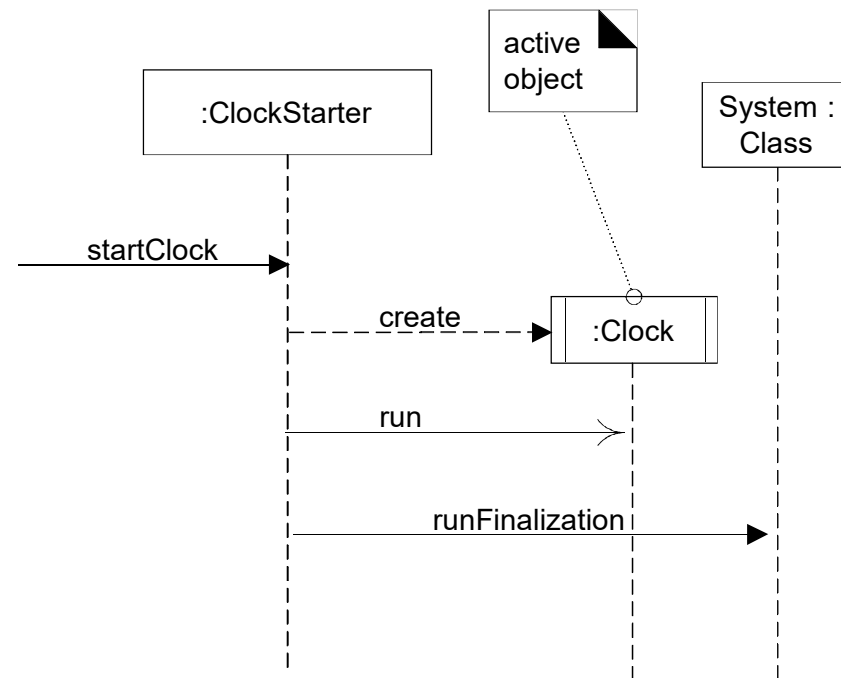
a **filled arrow** is the more common **synchronous call**

In Java, for example, an asynchronous call may occur as follows:

```
// Clock implements the Runnable interface
Thread t = new Thread( new Clock() );
t.start();
```

the asynchronous *start* call always invokes the *run* method on the *Runnable (Clock)* object

to simplify the UML diagram, the *Thread* object and the *start* message may be avoided (they are standard "overhead"); instead, the essential detail of the *Clock* creation and the *run* message imply the asynchronous call



Chapter 16. UML Class Diagrams

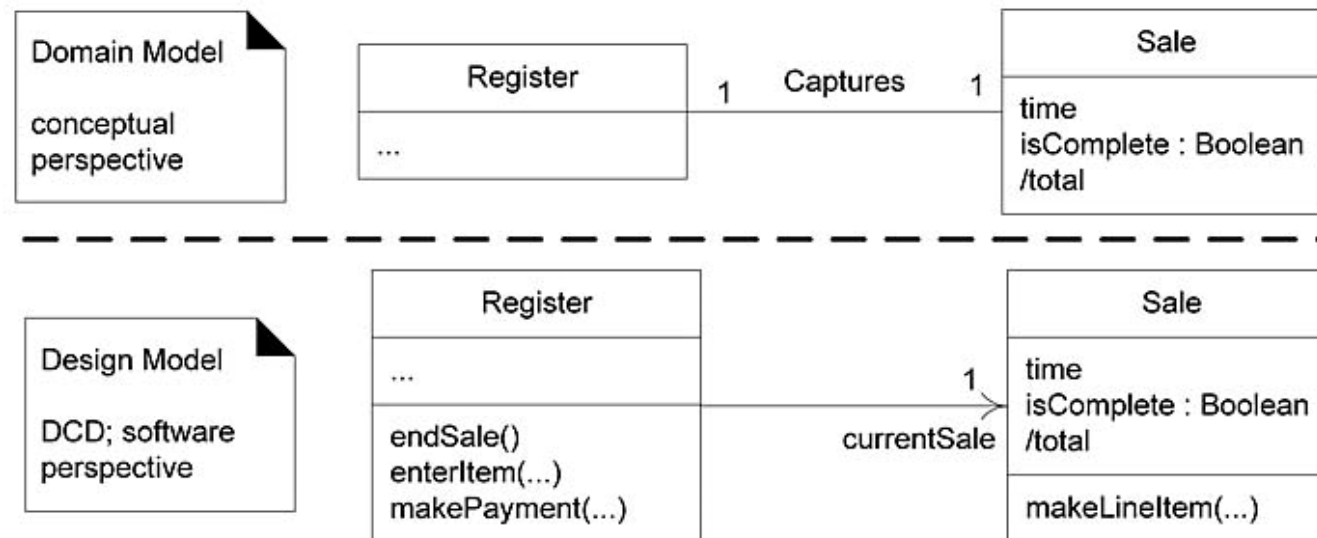
“To iterate is human, to recurse, divine.”

Objectives

- Provide a reference for frequently used UML class diagram notation.

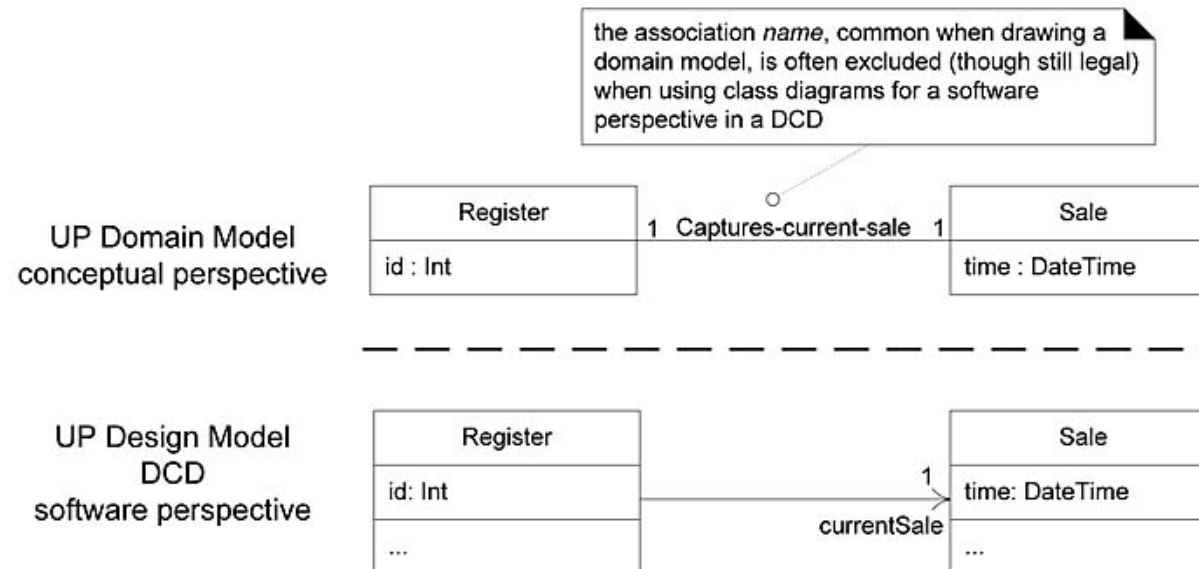
Design Class Diagram

- The same UML class diagrams can be used in multiple perspectives.
 - In a conceptual perspective, **Domain model**
 - In a design perspective, **Design Class Diagram (DCD)**



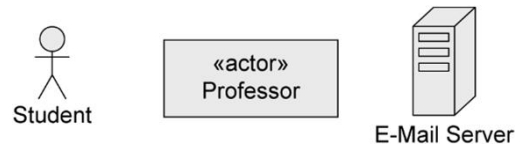
Associations in Different Perspectives

- Domain model :
 - Show association names but avoid navigation arrows since a domain model is not a software perspective.
- DCD :
 - Follow the UML specification



Keywords

- Textual adornment to categorize a model element
 - e.g. keyword to categorize that a classifier box is an interface is «interface»
 - e.g. «actor» keyword used in use-case diagrams: replace the human stick-figure actor icon with a class box to model computer-system or robotic actors

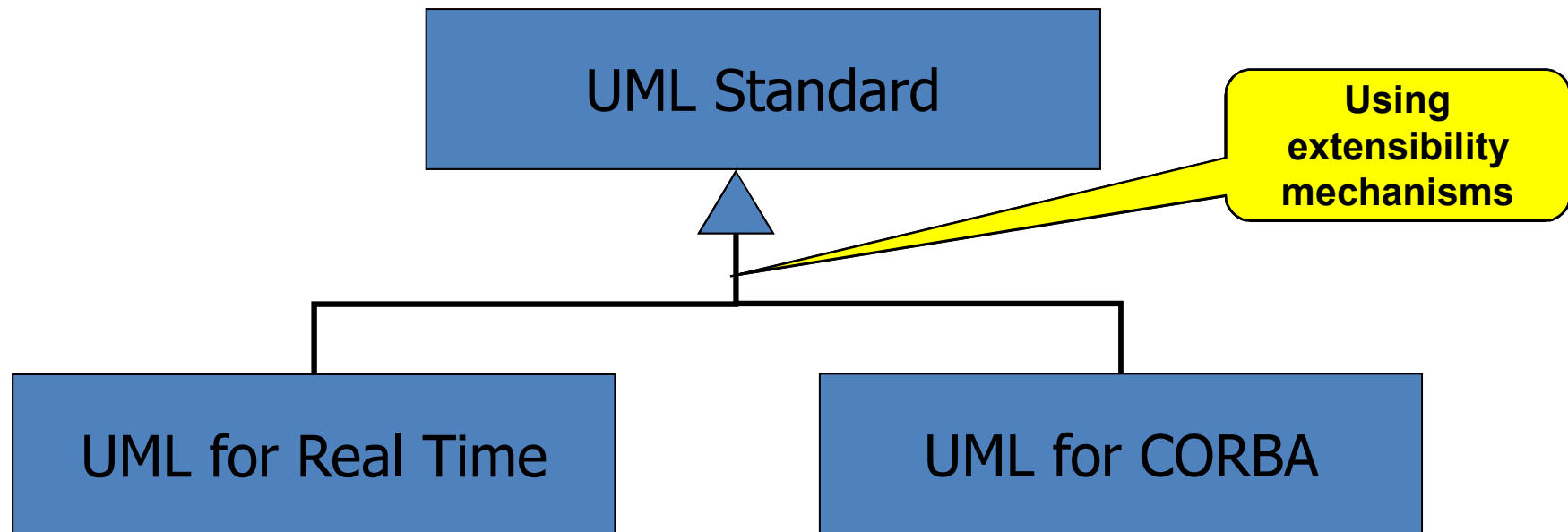


- Most keywords are shown in guillemet (« ») but some are shown in curly braces, such as {abstract}

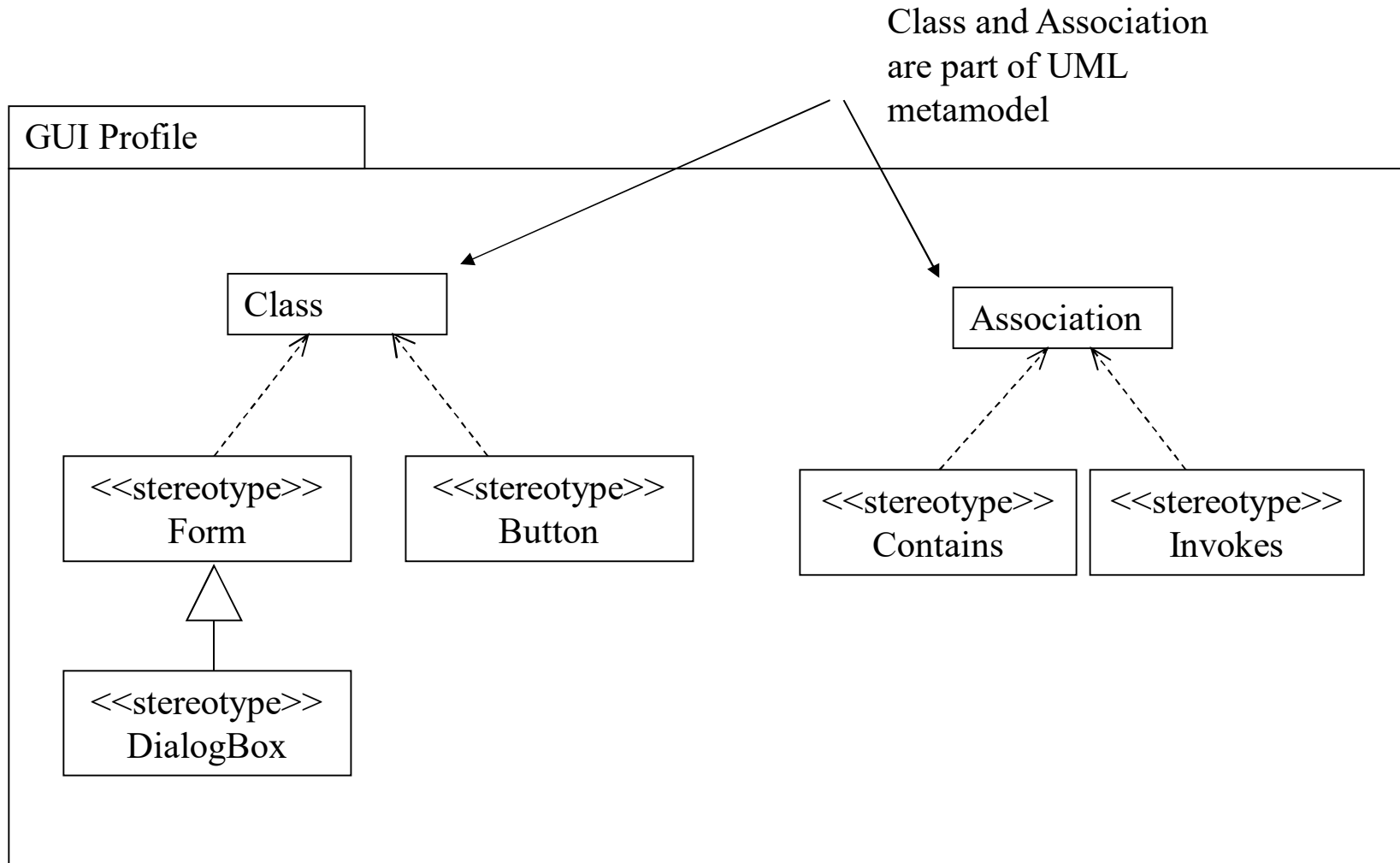
Keyword	Meaning	Example Usage
«actor»	classifier is an actor	in class diagram, above classifier name
«interface»	classifier is an interface	in class diagram, above classifier name
{abstract}	abstract element; can't be instantiated	in class diagrams, after classifier name or operation name
{ordered}	a set of objects have some imposed order	in class diagrams, at an association end

Stereotypes, Profiles and Tags

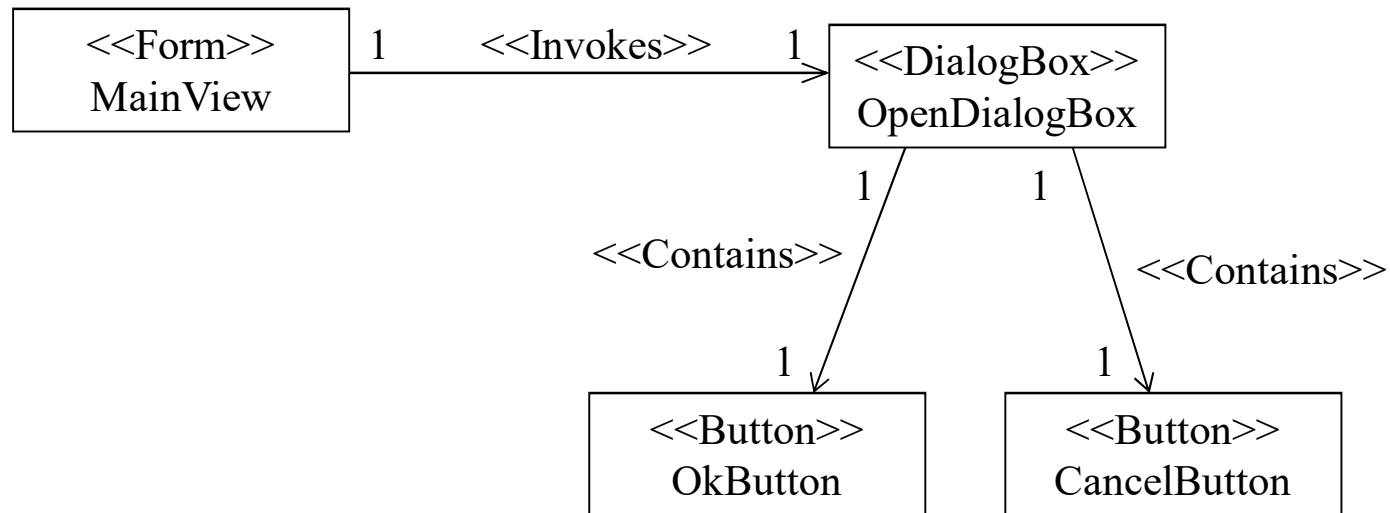
- A **stereotype** represents a refinement of an existing modeling concept.
 - defined within a **UML profile**, and is not a keyword.
 - provide an extension mechanism in the UML.
 - **UML profile**
 - Informally, a collection of related **stereotypes**, **tags**, and **constraints** to specialize the use of the UML for a **specific domain or platform**.
 - e.g, a UML profile for Real-time systems or for CORBA



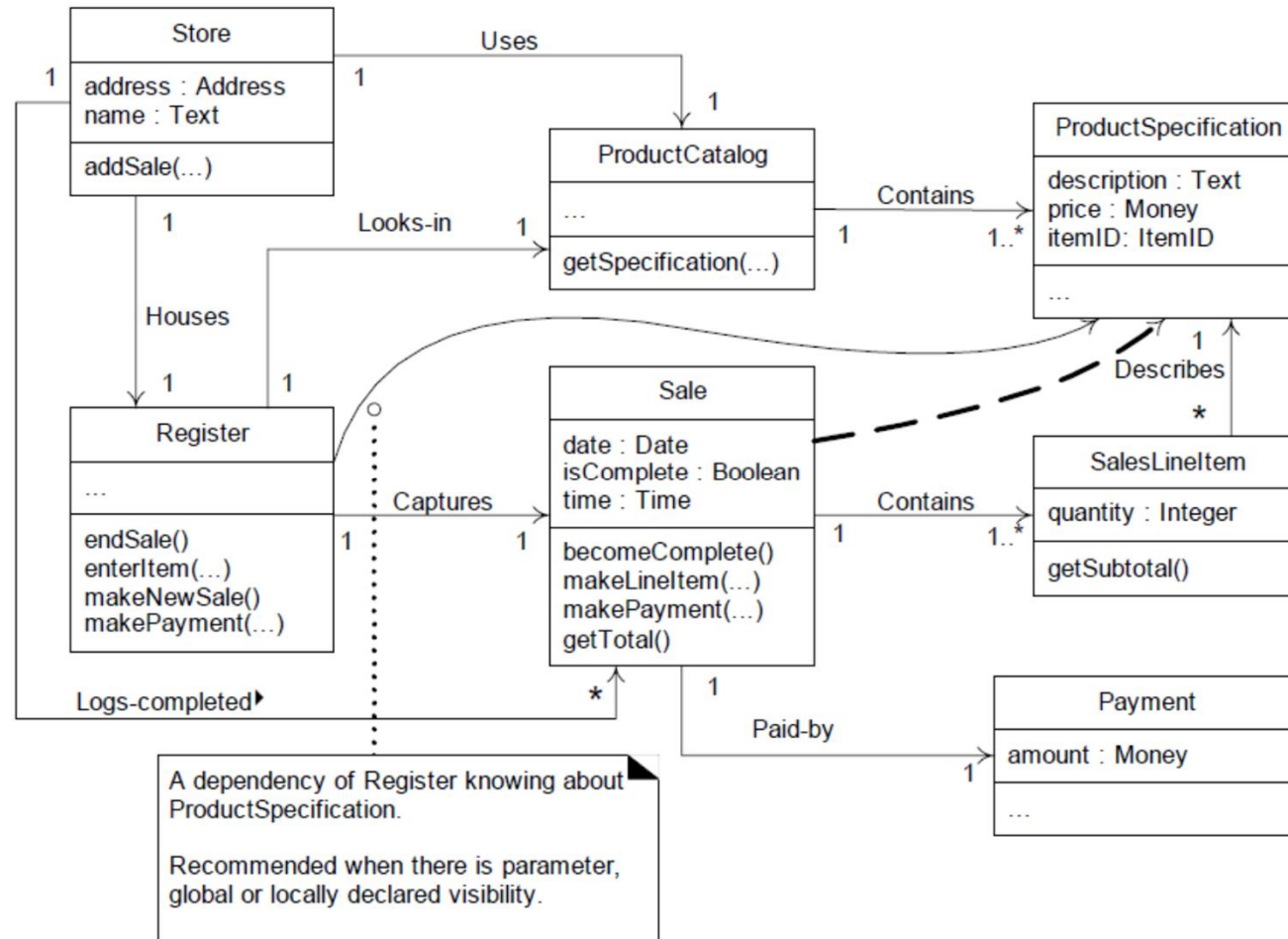
UML Profile Example: GUI profile package



UML Profile Example: Using the GUI profile



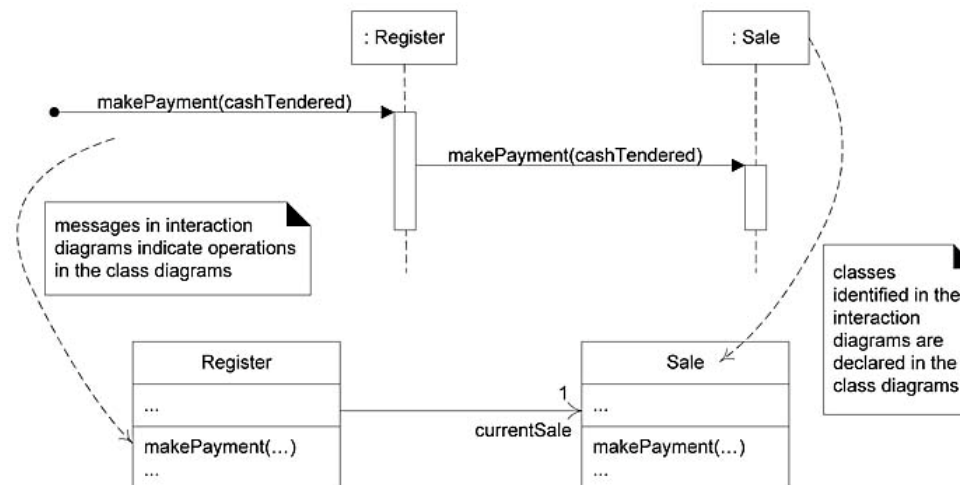
Example: Design Class Diagram



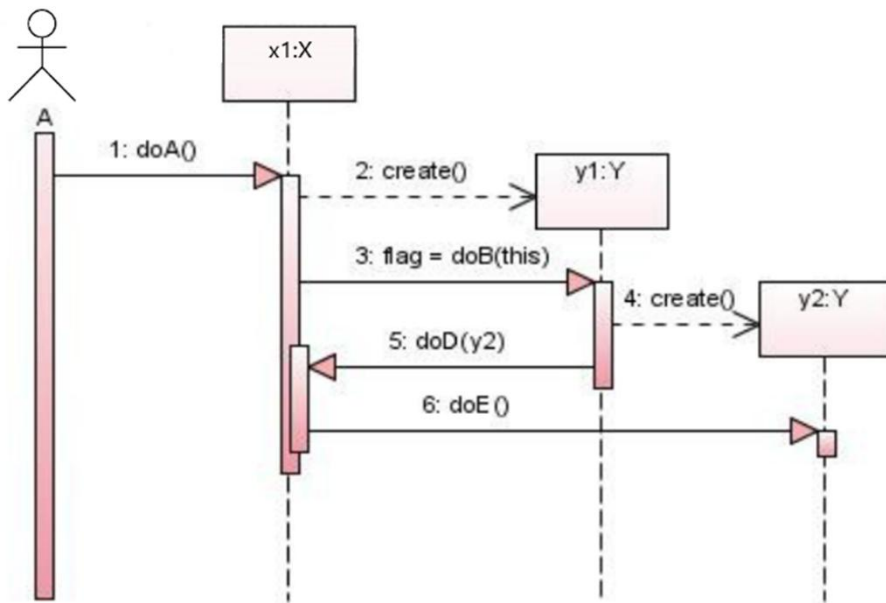
Dependency relationships indicating non-attribute visibility

What's the Relationship between Interaction and Class Diagrams?

- From interaction diagrams, the definitions of class diagrams can be generated.
 - When we draw interaction diagrams, a set of classes and their methods emerge.
 - e.g. if we started with the *makePayment* sequence diagram, we see that a *Register* and *Sale* class definition in a class diagram can be obviously derived.
 - Advice: Draw interaction diagrams before class diagrams.
 - But in practice, especially when following the agile modeling, these complementary dynamic and static views are drawn concurrently.



Deriving Code from Interaction Diagrams



```
public class Y {
    boolean doB(X x) {
        Y y2 = new Y();
        x.doD(y2);
        // ...
        return SomeBooleanValue;
    }
    void doE () {
        // ...
    }
    // NO doD definition here!!!
}
```